

Documento de reflexión

TC2038.601: Análisis y diseño de algoritmos avanzados

Fátima Montserrat Herrera González | A01770839

Kiara Hermann San Lucas | A01771209

Para la primera parte se utilizó el algoritmo de Knuth-Morris-Pratt (KMP), que es ideal para la búsqueda de patrones. Con este algoritmo se logra determinar si un patrón de código malicioso (mcode) se encuentra dentro de un archivo principal de transmisión (transmissionx), se obtiene su posición inicial sin necesidad de tener que volver a evaluar caracteres en la transmisión cuando ocurre una desalineación. Tenemos como resultado una complejidad temporal de $O(n + m)$ y espacial de $O(m)$.

Este proceso se divide en dos etapas:

- Construcción de la tabla LPS, donde se analiza el patrón de mcode para construir el arreglo que almacena la longitud del prefijo más largo que al mismo tiempo es un sufijo propio en cada subcadena del patrón (tomando un tiempo de $O(m)$ y espacio $O(m)$)
- Búsqueda de transmisión, donde se recorre el transmissionx y se ocupa LPS para no tener que volver a revisar completamente si concuerdan o no (siendo de complejidad $O(n)$).

En la segunda parte se utilizó el algoritmo de Manacher, el cual nos permite encontrar el palíndromo contiguo más largo en un archivo de transmisión en tiempo lineal. Tanto la complejidad temporal como la espacial son $O(n)$.

Este algoritmo se divide en los siguiente pasos:

- Normalizar el texto, se intercalan caracteres entre cada carácter del texto y se delimitan los extremos, de esta manera podemos tratar por igual a los palíndromos de longitud par e impar. Nuestra cadena resulta ser de longitud $2n + 3$.
- Explorar simetría y su expansión, se mantiene un registro en el centro actual y el límite derecho del palíndromo que más lejos se logra extender hacia la derecha. Para cada nueva posición, si esta se encuentra dentro de nuestro límite conocido, se reutiliza el

radio previamente calculado de su posición espejo para evitar tener que volver a comparar.

- Mapear índices, cuando se encuentra un radio máximo, al terminar de repasar toda la cadena, se convierten las coordenadas en la cadena transformada a los índices originales en base 1.

Finalmente para la parte tres, optamos por hacer uso del Suffix Array + LPC, para localizar la subcadena en común más larga. Aquí lo que estamos haciendo es combinar tres algoritmos distintos.

Primero tenemos la construcción del Suffix Array y para optimizar usamos la técnica de duplicación de prefijos, por lo que en cada ronda se ordenan los sufijos considerando el doble de caracteres que en la ronda anterior, usando un counting sort para que cada ronda sólo sea $O(n)$. Y como hacen falta $O(\log n)$ rondas para cubrir un texto de tamaño n , terminamos con una complejidad total de $O(n \log n)$ para esta primera parte.

Después cuando construimos el LPC usando el algoritmo de Kasai, lo que hacemos es guardar para cada par de sufijos consecutivos que haya en el Suffix Array cuántos caracteres tienen en común al inicio. Lo interesante es que el algoritmo de Kasai nos permite no tener que volver a recalcular desde cero cada vez que se avanza en el arreglo, logrando así una complejidad de $O(n)$.

Por último para encontrar la subcadena común más larga entre los dos textos, se concatenan los dos textos utilizando separadores, se construye el Suffix Array y el LPC de la concatenación, posteriormente recorreremos el arreglo LPC buscando el valor máximo entre sufijos que provengan de textos distintos, y ese valor máximo es la subcadena común más larga. Este último recorrido tiene una complejidad de $O(n)$.

Por lo que en total la parte tres tiene una complejidad final de $O(n \log n)$.

Concluimos que se realizó una elección óptima de los algoritmos necesarios para la resolución de esta situación problema, garantizando una ejecución fluida, consumo de memoria acotado, logrando obtener una complejidad temporal total de $O(u \log u)$, siendo $u = n + m$, y una complejidad espacial total de $O(n_1 + n_2)$, recordando que esto es mejor que un enfoque de fuerza bruta con complejidades superiores y tiempos de ejecución ineficientes.